

Documento Técnico Completo — Plataforma REVISA

1. Objetivo do documento

Este documento consolida a arquitetura funcional, lógica, técnica e de entrega da Plataforma REVISA, considerando a separação correta entre:

- REVISA como instituição gestora
- POLOS como unidades operacionais
- GABINETES como estruturas político-operacionais dos vereadores parceiros
- VEREADORES como atores institucionais com visão executiva própria
- EQUIPES DE RUA e EQUIPES POLÍTICAS como canais de captação territorial
- BENEFICIÁRIOS como pessoas atendidas pelos polos
- CIDADÃOS, APOIADORES e LIDERANÇAS como pessoas captadas em contexto territorial/político-social
- ADMINISTRAÇÃO CENTRAL REVISA como camada de controle irrestrito e governança

O desenho proposto busca garantir:

- modularidade
- baixa ambiguidade de domínio
- segurança por escopo
- crescimento sem refatoração estrutural precoce
- compatibilidade com BI, georreferenciamento e IA futura

2. Princípios arquiteturais

1. Separação de contextos de negócio

Cada domínio deve ter responsabilidade clara: gestão institucional, operação do polo, captação territorial, governança, analytics.

2. Monólito modular como estratégia inicial

O sistema deve nascer como um monólito modular bem organizado, com fronteiras explícitas por domínio, evitando microserviços prematuros.

3. Escopo de acesso por vínculo organizacional

A autorização não será apenas por perfil, mas também por vínculo com REVISA, POLO, GABINETE, EQUIPE e VEREADOR.

4. Cadastro mestre desacoplado do uso operacional

Pessoa, papel, usuário e vínculo organizacional não são a mesma entidade.

5. App mobile como canal de entrada, não como dono da verdade cadastral

O mobile capta, sincroniza e encaminha; a consolidação ocorre no núcleo transacional.

6. BI e analytics fora do fluxo transacional pesado

Relatórios estratégicos devem sair de views/materializações e camada analítica.

7. Preparação para IA futura sem contaminar o core

O sistema deve expor eventos, histórico e dados governados para futuros modelos, sem acoplar IA às rotinas centrais desde o início.

3. Contextos de negócio e fronteiras

3.1 Domínio institucional REVISA

Responsável por:

- gestão central da plataforma
- cadastro de usuários
- gestão de perfis e vínculos
- visão global de polos, gabinetes, vereadores, equipes e beneficiários
- relatórios globais
- governança e auditoria

3.2 Domínio Polo

Responsável por:

- beneficiários do polo
- modalidades ofertadas
- atividades locais
- frequências
- registros diários
- ocorrências
- pedidos de compra/material
- tarefas operacionais do polo

3.3 Domínio Gabinete / Mandato

Responsável por:

- equipes políticas e de rua
- cadastros captados via mobile
- apoiadores, lideranças e contatos territoriais
- demandas políticas/territoriais
- ações de campo
- relatórios operacionais do gabinete

3.4 Domínio Vereador

Responsável por visão executiva do seu ecossistema:

- polos vinculados
- beneficiários vinculados ou encaminhados em seu contexto

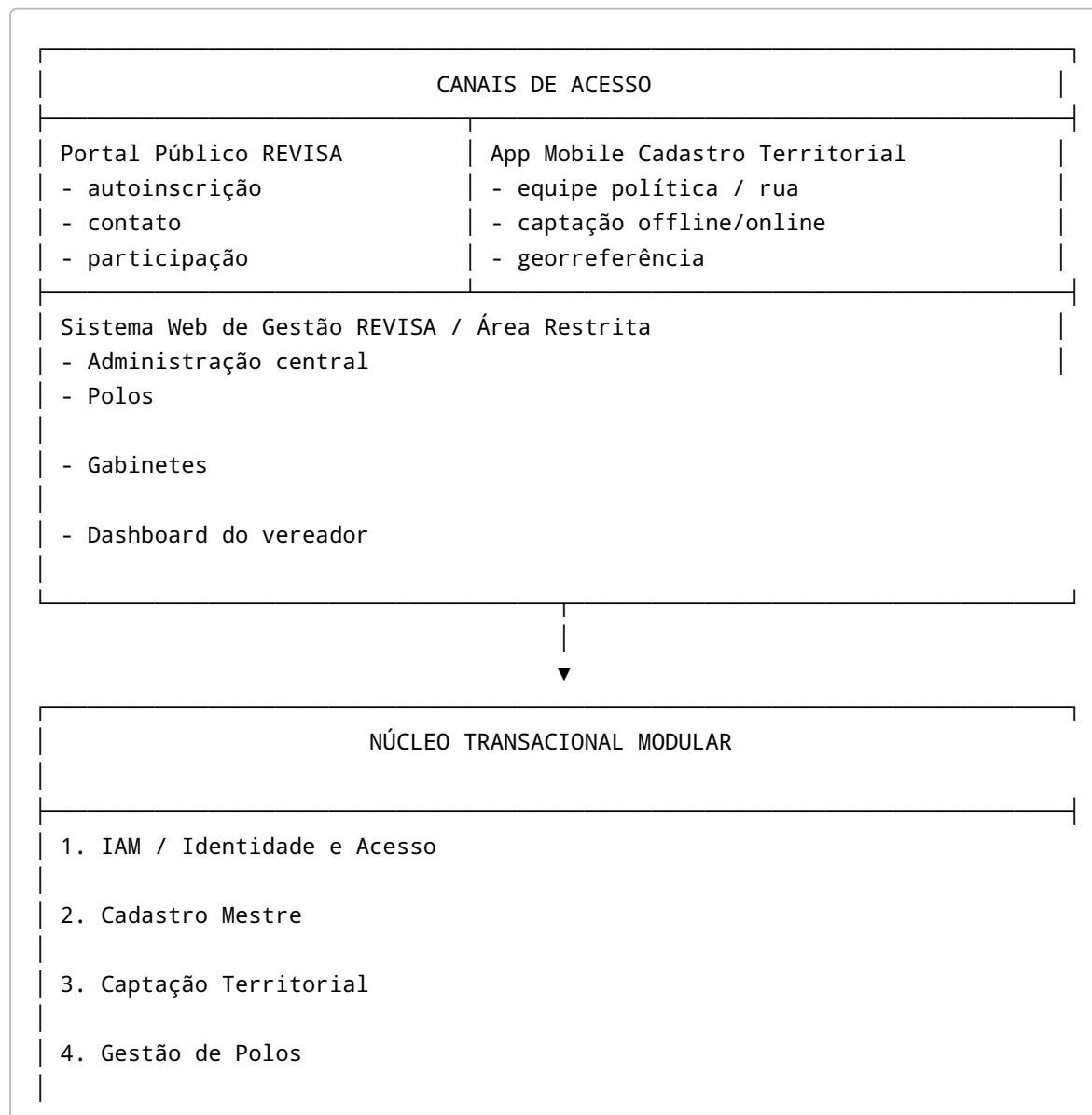
- eventos
- demandas
- tarefas
- mapa territorial
- acompanhamento de equipe política

3.5 Domínio Público

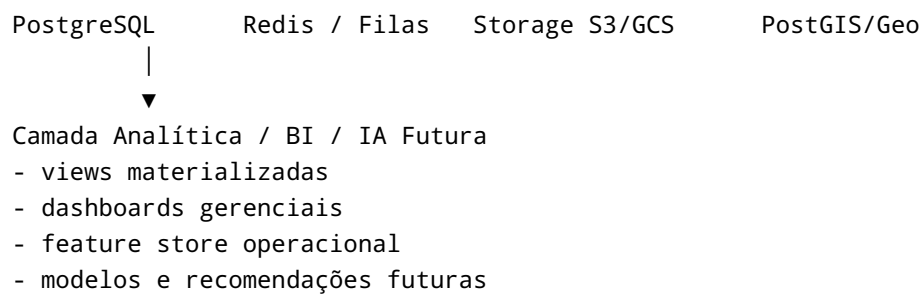
Responsável por:

- autoinscrição de voluntários
- formulário público de interesse
- contato institucional
- páginas públicas da REVISA

4. Diagrama em texto — arquitetura macro



- 5. Gestão de Gabinetes
- 6. Dashboard do Vereador
- 7. Eventos, Agendas e Atividades
- 8. Parcerias e Relacionamento Institucional
- 9. LGPD, Auditoria e Governança
- 10. Relatórios Operacionais



5. Diagrama em texto — módulos e interações

[Portal Público]

- cria autoinscrição de voluntário
- cria pré-cadastro de interessado
- consulta informações públicas

[App Mobile Cadastro]

- cria contato territorial
- cria pré-beneficiário
- cria apoiador/liderança
- registra demanda territorial
- sincroniza com Núcleo de Captação

[Núcleo de Captação Territorial]

- valida duplicidade
- classifica cadastro
- vincula gabinete / vereador / equipe
- encaminha para triagem
- gera evento para Dashboard e BI

[Núcleo de Polos]

- └─ recebe beneficiário validado
- └─ vincula ao polo
- └─ registra modalidade
- └─ registra frequência
- └─ registra ocorrência
- └─ registra tarefa e pedido
- └─ publica dados operacionais

[Núcleo de Gabinetes]

- └─ acompanha equipe de rua
- └─ acompanha contatos captados
- └─ acompanha tarefas territoriais
- └─ acompanha demandas
- └─ publica consolidados do mandato

[Dashboard do Vereador]

- └─ consome indicadores do gabinete
- └─ consome indicadores dos polos vinculados
- └─ exibe mapa, tarefas, urgências e relatórios
- └─ respeita isolamento por vereador

[Administração REVISA]

- └─ cadastra usuários
- └─ define perfis e vínculos
- └─ administra polos, gabinetes e vereadores
- └─ consulta logs
- └─ enxerga toda a operação

6. Arquitetura funcional por módulo

6.1 Módulo IAM — Identidade e Acesso

Responsabilidades:

- autenticação por login e senha
- refresh token
- reset de senha
- ativação/desativação de usuário
- RBAC por perfil
- escopo por organização/vínculo
- auditoria de acesso

6.2 Módulo Cadastro Mestre

Responsabilidades:

- cadastro base de pessoas
- cadastro de organizações

- cadastro de endereços
- cadastro de documentos
- deduplicação
- consentimentos e vínculo de titularidade

6.3 Módulo de Captação Territorial

Responsabilidades:

- entrada mobile
- sincronização
- contatos de campo
- apoiadores
- lideranças
- pré-beneficiários
- demandas comunitárias
- classificação e encaminhamento

6.4 Módulo de Gestão dos Polos

Responsabilidades:

- gestão local do polo
- beneficiários do polo
- modalidades
- agenda local
- frequência
- ocorrências
- registros diários
- tarefas
- compras/materiais

6.5 Módulo de Gestão dos Gabinetes

Responsabilidades:

- equipes políticas
- supervisão de cadastrantes
- acompanhamento de demandas
- produção territorial
- indicadores da equipe
- ações prioritárias e urgentes

6.6 Módulo Dashboard do Vereador

Responsabilidades:

- visão executiva exclusiva
- consolidação por polo
- consolidação por equipe política
- mapa de polos e cobertura
- tarefas abertas e concluídas

- relatórios do mandato dentro do ecossistema REVISA

6.7 Módulo Eventos, Agendas e Atividades

Responsabilidades:

- agenda institucional
- agenda do polo
- agenda do gabinete
- eventos pontuais
- atividades recorrentes
- presença e participação

6.8 Módulo Parcerias e Relacionamento Institucional

Responsabilidades:

- empresas parceiras
- doações e contrapartidas
- histórico de relacionamento
- contratos e documentos

6.9 Módulo BI / Analytics / IA futura

Responsabilidades:

- indicadores gerenciais
- dashboards executivos
- relatórios estratégicos
- base para georreferenciamento
- feature store operacional
- recomendação futura

6.10 Módulo LGPD e Auditoria

Responsabilidades:

- consentimentos
- revogação
- trilha de acesso
- trilha de alteração
- trilha de exportação
- retenção de dados
- anonimização

7. Matriz de perfis e escopos

7.1 Perfis primários

- ADM_GERAL_REVISA

- ADM_REVISA
- VEREADOR
- CHEFE_GABINETE
- SUPERVISOR_EQUIPE_POLITICA
- ADM_POLO
- COORDENADOR_POLO
- COLABORADOR_POLO
- COLABORADOR_GABINETE
- COLABORADOR_REVISA
- BENEFICIARIO
- EMPRESA_PARCEIRA
- VOLUNTARIO_AUTOINSCRITO

7.2 Escopos possíveis

- GLOBAL
- REVISA
- VEREADOR
- GABINETE
- POLO
- EQUIPE
- PRÓPRIO_REGISTRO

7.3 Matriz resumida de acesso

Perfil	Escopo	Cadastros	Beneficiários	Polos	Gabinetes
ADM_GERAL_REVISA	GLOBAL	total	total	total	total
ADM_REVISA	REVISA/GLOBAL delegado	total	total	total	leitura ampla
VEREADOR	VEREADOR	próprios	próprios/vinculados	vinculados	próprio
CHEFE_GABINETE	GABINETE	próprios	encaminhados do gabinete	vinculados	próprio
SUPERVISOR_EQUIPE_POLITICA	EQUIPE/GABINETE	da equipe	encaminhados da equipe	leitura vinculada	leitura do gabinete
ADM_POLO	POLO	do polo	total do polo	próprio polo	não
COORDENADOR_POLO	POLO	leitura/edição do polo	total do polo	próprio polo	não
COLABORADOR_POLO	POLO	parcial	operacional do polo	leitura do polo	não

Perfil	Escopo	Cadastros	Beneficiários	Polos	Gabinetes
COLABORADOR_GABINETE	GABINETE/EQUIPE	próprios	encaminhados pela equipe	leitura vinculada	próprio
COLABORADOR_REVISA	REVISA	conforme função	conforme função	conforme função	leitura se autorizado
BENEFICIARIO	PRÓPRIO_REGISTRO	próprio	próprio	agenda autorizada	não
EMPRESA_PARCEIRA	PRÓPRIO_REGISTRO/ CONTEXTUAL	próprio	não	eventos/ ações vinculadas	não
VOLUNTARIO_AUTOINSCRITO	PRÓPRIO_REGISTRO	próprio	não	não	não

7.4 Regras-chave de autorização

- nenhum vereador vê dados de outro vereador
- nenhum gabinete vê contatos de outro gabinete
- nenhum polo vê beneficiários de outro polo, salvo permissão global
- dados públicos, políticos e assistenciais devem permanecer segregados por finalidade
- administração central vê tudo com trilha obrigatória de auditoria

8. DER inicial — modelo entidade-relacionamento conceitual

```

ORGANIZATION
├─ id
├─ type [REVISA, POLO, GABINETE, EMPRESA]
├─ parent_organization_id
└─ ...

```

```

PERSON
├─ id
├─ dados civis e contato
└─ ...

```

```

USER
├─ id
├─ person_id -> PERSON
├─ credenciais
└─ ...

```

```

ROLE
PERMISSION
USER_ROLE
ROLE_PERMISSION

```

VEREADOR
├─ id
├─ person_id -> PERSON
├─ gabinete_organization_id -> ORGANIZATION
└─ ...

TEAM
TEAM_MEMBER

PERSON_LINK
├─ person_id -> PERSON
├─ organization_id -> ORGANIZATION
├─ vereador_id -> VEREADOR
├─ link_type [BENEFICIARIO, APOIADOR, LIDERANCA, COLABORADOR, VOLUNTARIO, PARCEIRO]
└─ ...

CONTACT_CAPTURE
├─ captured_by_user_id -> USER
├─ vereador_id -> VEREADOR
├─ classification [BENEFICIARIO, APOIADOR, LIDERANCA, CIDADAO]
└─ ...

POLO_BENEFICIARIO
├─ polo_organization_id -> ORGANIZATION
├─ person_id -> PERSON
└─ ...

MODALIDADE
MATRICULA_MODALIDADE
FREQUENCIA
OCORRENCIA
DAILY_LOG
PURCHASE_REQUEST

TASK
EVENT
AGENDA_ITEM
DEMAND
PARTNERSHIP
ATTACHMENT
CONSENT
AUDIT_LOG
ACCESS_LOG
EXPORT_LOG
ADDRESS
GEO_ENTITY

9. DER inicial — entidades prioritárias do MVP

9.1 Núcleo de acesso

- users
- roles
- permissions
- user_roles
- role_permissions
- user_scope_assignments

9.2 Núcleo institucional

- organizations
- vereadores
- teams
- team_members

9.3 Núcleo de cadastro

- persons
- addresses
- person_links
- consents
- duplicate_review_queue

9.4 Núcleo mobile e gabinete

- contacts_capture
- gabinete_actions
- gabinete_tasks
- gabinete_reports

9.5 Núcleo de polo

- polos_beneficiarios
- modalidades
- matriculas_modalidade
- frequencias
- ocorrencias
- daily_logs
- purchase_requests

9.6 Núcleo transversal

- tasks
- events
- demands
- attachments
- geo_layers
- audit_logs
- access_logs

- export_logs
-

10. Estrutura de banco proposta

10.1 Estratégia de persistência

- **PostgreSQL** como banco principal
- **PostGIS** para georreferenciamento
- **Redis** para cache, filas leves e rate limiting
- possibilidade de réplica analítica ou schema analítico separado

10.2 Estratégia de schemas lógicos

Para evitar um banco sem governança, recomenda-se organizar em schemas lógicos:

- iam
- core
- territory
- gabinete
- polo
- events
- governance
- analytics

10.3 Tabelas iniciais essenciais

IAM

- iam.users
- iam.roles
- iam.permissions
- iam.user_roles
- iam.role_permissions
- iam.user_scope_assignments
- iam.refresh_tokens

Core

- core.persons
- core.addresses
- core.organizations
- core.person_links
- core.consent
- core.attachments

Territory / Gabinete

- territory.contacts_capture
- territory.demands
- territory.territorial_actions

- territory.leadership_signals

Polo

- polo.beneficiarios
- polo.modalidades
- polo.matriculas
- polo.frequencias
- polo.ocorrencias
- polo.daily_logs
- polo.purchase_requests

Events

- events.events
- events.activities
- events.calendar_items
- events.participations

Governance

- governance.audit_logs
- governance.access_logs
- governance.export_logs
- governance.privacy_requests

Analytics

- analytics.fact_captures
- analytics.fact_attendance
- analytics.fact_demands
- analytics.dim_polo
- analytics.dim_vereador
- analytics.dim_team
- analytics.dim_date

11. Estrutura de APIs

11.1 Estratégia

- API única versionada: `/api/v1`
- contrato REST para operações transacionais
- eventos internos assíncronos para integrações internas
- autenticação JWT
- autorização por RBAC + escopo

11.2 Agrupamento final por domínio

```
/api/v1/auth  
/api/v1/users
```

```
/api/v1/roles
/api/v1/organizations
/api/v1/vereadores
/api/v1/teams
/api/v1/persons
/api/v1/consents
/api/v1/contacts-capture
/api/v1/demands
/api/v1/polos
/api/v1/polos/{id}/beneficiarios
/api/v1/polos/{id}/modalidades
/api/v1/polos/{id}/frequencias
/api/v1/polos/{id}/ocorrencias
/api/v1/polos/{id}/daily-logs
/api/v1/polos/{id}/purchase-requests
/api/v1/gabinetes
/api/v1/gabinetes/{id}/acoes
/api/v1/gabinetes/{id}/tarefas
/api/v1/vereadores/{id}/dashboard
/api/v1/events
/api/v1/activities
/api/v1/tasks
/api/v1/reports
/api/v1/geo
/api/v1/audit
/api/v1/privacy
```

11.3 Exemplos de endpoints prioritários

Auth

- POST /api/v1/auth/login
- POST /api/v1/auth/refresh
- POST /api/v1/auth/logout
- POST /api/v1/auth/forgot-password

Usuários e papéis

- POST /api/v1/users
- GET /api/v1/users/{id}
- PATCH /api/v1/users/{id}
- POST /api/v1/users/{id}/roles
- POST /api/v1/users/{id}/scopes

Cadastro mestre

- POST /api/v1/persons
- GET /api/v1/persons/{id}
- GET /api/v1/persons?cpf=&name=
- POST /api/v1/consents
- POST /api/v1/persons/{id}/links

Captação mobile

- POST /api/v1/contacts-capture
- GET /api/v1/contacts-capture?vereador_id=&team_id=
- POST /api/v1/contacts-capture/{id}/classify
- POST /api/v1/contacts-capture/{id}/forward-to-polo

Polo

- POST /api/v1/polos
- GET /api/v1/polos/{id}
- GET /api/v1/polos/{id}/beneficiarios
- POST /api/v1/polos/{id}/beneficiarios
- POST /api/v1/polos/{id}/frequencias
- POST /api/v1/polos/{id}/ocorrencias
- POST /api/v1/polos/{id}/daily-logs
- POST /api/v1/polos/{id}/purchase-requests

Gabinete

- GET /api/v1/gabinetes/{id}/captacoes
- GET /api/v1/gabinetes/{id}/demandas
- GET /api/v1/gabinetes/{id}/tarefas
- POST /api/v1/gabinetes/{id}/acoes

Vereador

- GET /api/v1/vereadores/{id}/dashboard
- GET /api/v1/vereadores/{id}/mapa
- GET /api/v1/vereadores/{id}/indicadores

Governança

- GET /api/v1/audit/logs
- GET /api/v1/privacy/consents
- POST /api/v1/privacy/requests

12. Estrutura final do repositório

A recomendação é consolidar a entrega em **um monorepo único**, com fronteiras claras por aplicação e pacote compartilhado, evitando múltiplos repositórios frágeis.

12.1 Estrutura sugerida

```
revisa-platform/  
├─ README.md  
├─ docs/  
│   └─ arquitetura/  
│   └─ negocio/
```

```

|   |   | api/
|   |   | banco/
|   |   | └─ governanca/
|   | └─ apps/
|   |   | api/
|   |   | web/
|   |   | └─ mobile/
|   | └─ packages/
|   |   | shared-types/
|   |   | ui/
|   |   | auth/
|   |   | └─ analytics-contracts/
|   | └─ infra/
|   |   | docker/
|   |   | nginx/
|   |   | terraform/
|   |   | github-actions/
|   |   | └─ monitoring/
|   | └─ database/
|   |   | migrations/
|   |   | seeds/
|   |   | schemas/
|   |   | └─ views/
|   | └─ scripts/

```

12.2 Aplicações

apps/api

- FastAPI
- SQLAlchemy/Alembic
- RBAC
- regras de negócio
- jobs e integrações

apps/web

- Next.js
- dashboard administrativo
- dashboard do vereador
- operação dos polos
- painel do gabinete

apps/mobile

- Flutter
- captação de campo
- pré-cadastro
- sincronização
- operação simplificada

12.3 Packages compartilhados

- contratos de payload
 - enums institucionais
 - design system
 - helpers de autenticação
 - tipos compartilhados
-

13. Decisão de codificação e estratégia de implementação

13.1 Diretriz

Não adotar codificação incremental excessivamente fragmentada.

A codificação deve ser feita por **blocos completos de domínio**, com fechamento funcional de ponta a ponta.

13.2 Unidades de entrega corretas

Cada sprint deve fechar um domínio completo, incluindo:

- banco
- API
- frontend correspondente
- regras de permissão
- logs de auditoria
- testes mínimos

Exemplo correto:

- Sprint de Captação Territorial fecha: tabela + endpoint + tela mobile + listagem web + permissão + auditoria

Exemplo incorreto:

- criar apenas tabela agora, tela depois, regra depois, permissão depois

13.3 Ordem ideal de implementação

1. fundação técnica e autenticação
 2. organizações, vereadores, gabinetes, polos e equipes
 3. cadastro mestre e consentimentos
 4. captação mobile + triagem
 5. operação dos polos
 6. gestão de gabinete
 7. dashboard vereador
 8. relatórios, mapas e governança ampliada
-

14. Backlog MVP por sprint

Sprint 0 — Fundação de plataforma

Objetivo: estabelecer base estável do repositório e infraestrutura.

Entregas:

- monorepo inicial
- docker compose
- pipelines CI/CD
- configuração de ambientes
- PostgreSQL + Redis
- API FastAPI inicial
- Web Next.js inicial
- Mobile Flutter inicial
- observabilidade básica
- logging estruturado
- convenções de branch e release

Sprint 1 — IAM completo

Objetivo: fechar autenticação e autorização.

Entregas:

- login/logout/refresh
- recuperação de senha
- usuários
- papéis
- permissões
- escopos
- middleware de autorização
- trilha de acesso
- seeds de perfis principais

Sprint 2 — Núcleo institucional

Objetivo: fechar estruturas organizacionais.

Entregas:

- organizations
- vereadores
- gabinetes
- polos
- teams
- team_members
- vínculo usuário-escopo
- telas web administrativas
- APIs completas

Sprint 3 — Cadastro mestre

Objetivo: consolidar base única de pessoas.

Entregas:

- persons
- addresses
- consents
- person_links
- busca/deduplicação inicial
- cadastro manual via web
- consulta por documento/nome/telefone
- auditoria de alteração cadastral

Sprint 4 — Mobile de captação territorial

Objetivo: fechar canal de entrada do campo.

Entregas:

- formulário mobile completo
- classificação de contato
- pré-beneficiário
- apoiador
- liderança
- demanda territorial
- sincronização com backend
- fila de envio
- listagem de captações no gabinete

Sprint 5 — Triagem e encaminhamento

Objetivo: transformar captação em operação válida.

Entregas:

- fila de triagem
- validação de duplicidade
- encaminhamento para polo
- conversão para beneficiário
- atribuição a gabinete/equipe
- auditoria do fluxo

Sprint 6 — Operação completa dos polos

Objetivo: fechar rotina mínima do polo ponta a ponta.

Entregas:

- beneficiários do polo

- modalidades
- matrículas
- frequência
- ocorrências
- registros diários
- tarefas operacionais
- pedidos de compra/material
- dashboard local do polo

Sprint 7 — Gestão do gabinete

Objetivo: fechar rotina da equipe política.

Entregas:

- painel do gabinete
- produção da equipe
- tarefas de campo
- demandas do gabinete
- ações urgentes/prioritárias
- relatórios da equipe política

Sprint 8 — Dashboard do vereador

Objetivo: visão executiva do mandato dentro da REVISA.

Entregas:

- indicadores do vereador
- polos ativos
- mapa com localizações
- quantidade de cadastrados por polo
- tarefas abertas/concluídas
- eventos e demandas
- ações prioritárias e urgentes
- visão isolada por vereador

Sprint 9 — Eventos, agenda e participação

Objetivo: fechar agenda institucional e operacional.

Entregas:

- eventos
- atividades recorrentes
- participação
- presença/frequência em atividade
- agenda do polo
- agenda do gabinete
- agenda institucional

Sprint 10 — Governança, relatórios e georreferenciamento

Objetivo: consolidar camada gerencial e de conformidade.

Entregas:

- audit logs
- export logs
- privacy requests
- relatórios gerenciais
- mapas por polo/vereador
- camadas geográficas
- painéis executivos REVISA

Sprint 11 — Fechamento MVP executivo

Objetivo: consolidar solução final implantável.

Entregas:

- revisão de performance
- revisão de segurança
- testes integrados
- hardening de permissões
- documentação de operação
- manual administrativo
- manual de uso por perfil
- pacote de deploy

15. Estrutura técnica recomendada por aplicação

15.1 Backend — FastAPI

```
apps/api/  
├── app/  
│   ├── main.py  
│   └── api/  
│       └── v1/  
│           ├── auth.py  
│           ├── users.py  
│           ├── organizations.py  
│           ├── vereadores.py  
│           ├── teams.py  
│           ├── persons.py  
│           ├── consents.py  
│           ├── contacts_capture.py  
│           ├── polos.py  
│           └── gabinete.py
```

```
├── vereador_dashboard.py
├── activities.py
├── events.py
├── tasks.py
├── reports.py
├── geo.py
├── audit.py
├── privacy.py
├── core/
├── models/
├── schemas/
├── services/
├── repositories/
├── events/
├── policies/
├── jobs/
├── alembic/
├── tests/
└── Dockerfile
```

15.2 Frontend Web — Next.js

```
apps/web/
├─ app/
│   ├── (public)/
│   ├── (auth)/
│   ├── (admin)/
│   ├── (polo)/
│   ├── (gabinete)/
│   ├── (vereador)/
│   └─ api/
├─ components/
├─ features/
│   ├── auth/
│   ├── users/
│   ├── organizations/
│   ├── persons/
│   ├── polos/
│   ├── gabinete/
│   ├── vereador/
│   ├── agenda/
│   ├── reports/
│   └─ governance/
├─ lib/
├─ hooks/
└─ styles/
```

15.3 Mobile — Flutter

```
apps/mobile/
├── lib/
│   ├── app/
│   ├── core/
│   ├── features/
│   │   ├── auth/
│   │   ├── captura/
│   │   ├── contatos/
│   │   ├── demandas/
│   │   ├── agenda/
│   │   └── sync/
│   ├── shared/
│   └── main.dart
├── test/
└── pubspec.yaml
```

16. Eventos de integração internos

Para desacoplamento lógico, mesmo no monólito modular, usar eventos internos como:

- user_created
- person_created
- contact_captured
- contact_classified
- beneficiary_forwarded_to_polo
- beneficiary_accepted_by_polo
- attendance_registered
- occurrence_opened
- task_created
- demand_prioritized
- event_created
- consent_revoked

Esses eventos servem para:

- alimentar analytics
 - disparar tarefas automáticas
 - produzir alertas
 - preparar futura evolução para serviços separados
-

17. Riscos e mitigação

17.1 Riscos de domínio

- mistura entre cadastro assistencial e cadastro político
- duplicidade de pessoas
- visão indevida entre vereadores
- acoplamento excessivo entre polo e gabinete

Mitigação:

- cadastro mestre
- links contextuais por papel
- autorização por escopo
- trilha de finalidade

17.2 Riscos técnicos

- dashboards lentos
- mobile dependente de rede
- explosão de complexidade por perfis
- monólito sem modularidade real

Mitigação:

- camada analítica separada
- sincronização offline-first progressiva
- política central de permissões
- organização em bounded contexts dentro do repositório

17.3 Riscos de governança

- uso indevido de dados sensíveis
- ausência de prova de acesso
- exportações descontroladas

Mitigação:

- audit logs
- export logs
- consentimentos
- mascaramento de dados
- revisão periódica de perfis

18. Conclusão executiva

A plataforma REVISA deve ser implementada como um **ecossistema modular unificado**, com três aplicações principais — **web, mobile e API** — sustentadas por um núcleo transacional bem modelado e uma camada analítica separada.

A chave do sucesso não é apenas tecnologia, e sim a correta separação entre:

- instituição
- operação territorial
- operação dos polos
- mandatos/gabinetes
- beneficiários
- contatos/apoiadores/lideranças
- governança e auditoria

A decisão mais sólida para este estágio é:

- **monorepo único**
- **monólito modular no backend**
- **sprints fechando domínios completos**
- **RBAC + escopo desde o primeiro dia**
- **cadastro mestre como núcleo**
- **camada analítica preparada para IA futura**

Esse desenho reduz retrabalho, evita ambiguidade institucional e deixa o sistema pronto para crescimento funcional, territorial e analítico.